CAMPUS DIGITAL FP

CFGS Desarrollo de Aplicaciones Multiplataforma

Programación de servicios y procesos

Entrega nº 1 - Tarea 04

03 / 03 / 2026

Carlota López-coronado Domingo

Índice

1. Ejercicio 4.1 2

2. Ejercicio 4.2 3

3. Ejercicio 4.3 4

Solución

1. Ejercicio 4.1

El primer ejercicio consiste en modificar el código del ejercicio 1 de la tarea anterior para permitir que varios clientes se conecten a un mismo servidor para trabajar de forma concurrente.

- Cambios en el programa :

El primer cambio que he realizado es incluir la función **servidor.accept()** dentro de un **while(true)** para que tras recibir una conexión, se mantenga en estado de escucha para permitir otras.

Después he creado una clase que extiende de **Thread** para que cada hilo tenga su propio contexto de ejecución. En ella he incluido la lógica del juego, incluyendo la variable numeroSecreto para que cada juego con cada cliente tenga un número independiente.

- Salida del programa :

```
carlotalcd@MacBook-Air-de-Carlota RandomNumMulClientes % java MultServidorRandomNum
Servidor escuchando en el puerto 2000
Cliente conectado desde: /127.0.0.1
Hilo Thread-0: Número secreto generado: 52
Cliente conectado desde: /127.0.0.1
Hilo Thread-1: Número secreto generado: 88
Hilo Thread-0 recibió: 3
Hilo Thread-1 recibió: 100
```

```
carlotalcd@MacBook-Air-de-Carlota RandomNumMulClientes % java ClienteRandomNum
Conectado al servidor de adivinanza en localhost:2000
Introduce un número (0-100): 3
Servidor dice: El número secreto es MAYOR
Introduce un número (0-100):
```

```
carlotalcd@MacBook-Air-de-Carlota RandomNumMulClientes % java ClienteRandomNum
Conectado al servidor de adivinanza en localhost:2000
Introduce un número (0-100): 100
Servidor dice: El número secreto es MENOR
Introduce un número (0-100):
```

2. Ejercicio 4.2

El segundo ejercicio consiste en modificar el código del ejercicio 2 de la tarea anterior (servidor de ficheros) para permitir la descarga concurrente de archivos por parte de múltiples usuarios.

- Cambios en el programa :

En primer lugar he incluido la lógica de lectura y envío de archivos en una clase interna llamada **HiloFichero** que hereda de Thread.

Al igual que en el ejercicio anterior, el **ServerSocket** se encuentra dentro de un bucle infinito, permitiendo aceptar nuevas peticiones de descarga mientras otras ya están en curso.

Por último, he usado una cadena de control (**FIN_FICHERO**) para que el cliente sepa exactamente cuándo termina la transmisión de un archivo de texto leído mediante BufferedReader.

- Salida del programa :

```
carlotalcd@MacBook-Air-de-Carlota Ficheros % java ServidorMultFicheros
Servidor de ficheros concurrente iniciado en el puerto 1500
Nuevo cliente conectado para descarga.
Hilo Thread-0 procesando: prueba.txt
Nuevo cliente conectado para descarga.
Hilo Thread-1 procesando: hola.txt
```

```
carlotalcd@MacBook-Air-de-Carlota Ficheros % java ClienteFicheros
Introduce el nombre del fichero que deseas descargar: prueba.txt
--- Contenido del fichero ---
Hola
Que tal
Esto es una prueba
-----------------------------
```

```
carlotalcd@MacBook-Air-de-Carlota Ficheros % java ClienteFicheros
Introduce el nombre del fichero que deseas descargar: hola.txt
--- Contenido del fichero ---
este
es
el
segundo archivo
DE PRUEBA
-----------------------------
```

3. Ejercicio 4.3

El tercer ejercicio consiste en crear un servidor de control avanzado que requiere autenticación de usuario y permite la gestión remota de ficheros mediante comandos específicos, operando de forma concurrente.

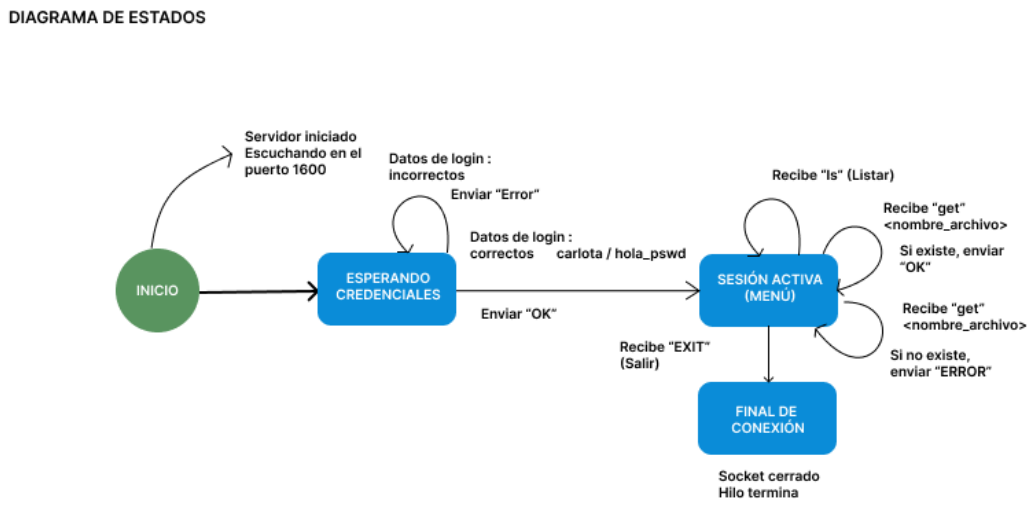
- Cambios en el programa :

He implementado una **lógica de control por fases**. El servidor comienza en un estado de "**Espera de Autenticación**" y solo permite el acceso al menú de comandos tras validar el usuario y contraseña (carlota/hola_pswd).

Además he diseñado un sistema de comunicación basado en palabras clave (**LS, GET, EXIT**). El servidor interpreta estas cadenas para decidir si debe listar el directorio, enviar un archivo o cerrar la conexión.

Por último ha incorporado el uso de la clase **file** y el método **.list()** para obtener dinámicamente el contenido de la carpeta actual del servidor y enviarlo al cliente.

- Diagrama de estados :



- Salida del programa :

Al ejecutar el sistema, el cliente solicita credenciales. Si son incorrectas, el servidor deniega el paso y solicita los datos de nuevo. Una vez logueado, el usuario puede listar los archivos del servidor (como prueba.txt) y visualizar su contenido en tiempo real. Al ser un sistema multihilo,

varios usuarios pueden estar navegando por el directorio del servidor simultáneamente sin interferencias.

```
Usuario :
carlota
Contraseña :
hola_pswd
Login correcto.

1. Listar | 2. Ver Archivo | 3. Salir
1
- ClienteFicheros.java
- .DS_Store
- hola.txt
- ClienteFicheros.class
- HiloControl.class
- ServidorMultFicheros.class
- ServidorMultFicheros.java
- prueba.txt
```

```
1. Listar | 2. Ver Archivo | 3. Salir
2
Nombre archivo: hola.txt
este
es
el
segundo archivo
DE PRUEBA

1. Listar | 2. Ver Archivo | 3. Salir
```